

企业级类 Figma AI 设计与应用生成引擎架构设计与实践项目介绍

需求背景介绍

在 AI 重塑软件生产力的今天，设计与开发之间的鸿沟依然是产品迭代的最大瓶颈之一。传统设计工作流中，设计师出图、前端还原的协作模式存在大量重复劳动，一个页面从设计稿到可运行代码往往需要数天时间；而且不同开发者还原度参差不齐，设计一致性难以保障。对于快速迭代的创业团队和中大型企业的产品部门来说，设计到代码的转化效率直接决定了产品上线速度。

妙码设计 AI 平台正是针对这一行业痛点打造的 **AI 驱动桌面端 D2C 设计生成引擎**，是一款**垂类专业化的设计智能体**。它的核心定位是：让大模型能力深度适配设计行业 workflow，从自然语言需求到设计产物再到代码输出全链路智能化，通过多智能体协作编排自动完成大部分设计生成与还原工作，只把创意决策和最终微调留给人类。

核心目标用户分层

1. **前端开发者**：全栈工程师、业务前端、低代码平台开发者。核心痛点是设计还原工作重复琐碎、占用大量业务开发时间，希望通过 AI 自动完成基础还原，把精力集中在业务逻辑和交互实现上。
2. **UI/UX 设计师**：产品设计师、运营设计师、独立设计师。核心痛点是从 0 到 1 出方案慢、改稿频繁、重复性设计多，希望通过 AI 快速生成多版方案，聚焦创意决策而非像素级调整。
3. **产品经理与创业者**：独立开发者、创业团队、产品经理。核心痛点是不会专业设计、找不到设计师、原型制作成本高，希望通过自然语言描述就能快速产出可用的设计稿和原型，加速需求验证。

4. **设计系统团队**：企业设计中台、组件库维护团队。核心痛点是设计规范落地难、各业务线还原度不一致、组件库更新同步成本高，希望通过标准化 MCP 接入让所有 AI 工具都遵循同一套设计资产。

垂类设计智能体的业务与技术价值

传统 AI 设计工具大多只是在单点做 AI 增强（比如一键生成图标、自动配色），没有从全链路重构设计生产流程。本项目作为**完整的设计生成智能体**，核心价值体现在：

业务价值

- **端到端自动化**：用户只需要输入自然语言需求，Agent 自动完成从需求拆解 → 区域规划 → 并行设计生成 → 视觉自验证 → 代码导出全流程，把页面产出周期从「天级」压缩到「分钟级」
- **人机协同最优分工**：Agent 承担 80% 重复性劳动（布局生成、样式还原、组件拼装、格式校验），人类只负责最后的 20% 创意调整和质量确认，真正做到「AI 提效 + 人类创意」
- **本地隐私保护**：所有设计文档和生成过程都在用户本地完成，不需要上传敏感的产品原型到云端，满足企业级隐私合规要求
- **可编辑可调整**：AI 生成不是终点而是起点，用户随时可以在类 Figma 编辑器中做精细化调整，兼顾自动化效率和创作灵活性，生成的设计产物完全可编辑、可分层、可复用

技术价值

- 验证了 **Codex CLI 本地智能体落地**：在真实的设计生成场景中，实践多智能体协作、Harness 内核、视觉自验证等复杂智能体架构设计
- 沉淀了 **MCP 标准化接入最佳实践**：设计资产通过标准 MCP 协议对外暴露，一次接入兼容 Codex / Claude Code / Cursor 等所有 MCP 客户端，解决 N×M 集成问题
- 完整工程化链路：覆盖 DSL 内核设计、编辑器开发、智能体引擎、MCP 服务、多端打包分发全流程，是学习 AI 桌面应用开发的绝佳实战项目

学习成果

需求分析与架构设计

- **深入理解 D2C 领域的技术路线选型**：系统掌握直接生成代码、中间层 DSL、组件库映射三条主流技术路线的核心差异与适用场景，清晰理解 DSL 中间层作为防腐层的架构价值，能够根据业务场景判断最优技术路线
- **掌握 AI 设计生成系统的架构设计方法论**：能够从需求拆解出发，完成编辑器内核、Agent 生成引擎、MCP 标准接入、多端交付四层架构的整体设计，具备分层职责划分、依赖方向管控、接口契约定义的专业架构思维

- **建立契约驱动开发的工程化认知**：深入理解 JSON Schema 在 AI 生成系统中的核心作用，掌握「Schema 定义 → 模型约束 → 运行时校验 → 失败重试 → 降级兜底」的完整防御式编程体系
- **掌握 Monorepo 全栈项目架构设计**：理解 packages 核心库层与 apps 应用层的职责划分，掌握单向依赖原则、按业务域拆分模块的架构思想
- **建立一人全栈 AI 化研发体系认知**：深入理解 AI 时代一人全栈研发的完整工作范式，掌握从需求发散到规格落地、从架构设计到代码实现的全流程 AI 提效方法

AI 引擎核心与多智能体实现

- **深入理解 Codex CLI 本地智能体架构**：掌握子进程调用模式、JSONL 事件流解析、Thread 持久化等核心机制，理解端侧 Agent 相比 SaaS API 的核心优势
- **掌握多智能体协作架构设计**：深入理解 Coordinator + 多 Collaborator 的角色分工模式，掌握任务按区域拆分、并行执行、结果汇总的编排策略
- **透彻掌握 Harness 内核四层架构**：理解记忆层、规划层、实施层、验证层的核心职责与数据流，建立智能体内核的架构设计思维
- **掌握视觉自验证闭环机制**：深入理解「截图捕获 → 多模态诊断 → 精准修复」的完整验证循环，掌握最小改动原则、修复次数上限、回归验证等稳定性保障机制
- **掌握 Generation Run 状态机设计**：理解从 queued 到 completed 的完整状态流转，掌握状态持久化、断点恢复、取消中断等工程能力

可视化编辑器开发

- **掌握类 Figma 编辑器的核心范式**：深入理解命令模式、不可变数据、发布订阅三大编辑器核心范式的原理与工程价值
- **掌握画布交互系统设计**：理解屏幕 / 视口 / 世界三层坐标的变换关系，掌握命中测试、选择集管理、reducer 状态机的实现原理
- **系统掌握 DSL 数据内核设计**：理解可辨识联合类型、正交属性拆分、原子化样式组合等 DSL 核心设计原则
- **掌握图层面板与属性面板的联动设计**：理解树形图层视图、选中状态同步、属性编辑实时预览的交互逻辑
- **建立设计变量系统认知**：理解 Design Variables 在设计系统中的核心作用，掌握颜色、字号、间距等设计 Token 的统一管理机制

MCP 服务端与跨进程通信

- **深入理解 MCP 协议的本质与价值**：掌握 MCP 与 Function Calling、LSP 的核心差异，理解工具、资源、提示三大协议能力的设计思想

- **具备 MCP Server 开发与工具注册能力**：能基于 @modelcontextprotocol/sdk 从零搭建标准 MCP Server，熟练使用 registerTool 注册工具、定义 Zod 输入 Schema
- **掌握跨进程 Bridge 桥接架构设计**：理解为什么需要 Bridge 层，掌握 Unix Domain Socket 与 Windows Named Pipe 的跨平台适配方案
- **掌握 Sidecar 模式的工程落地**：理解 Sidecar 独立进程的架构价值，掌握 macOS / Windows 双平台二进制构建策略
- **建立只读 MCP 的安全设计思维**：理解为什么设计资产接入优先做只读，掌握错误分级、优雅降级、超时控制等可靠性保障机制

学习产物

多智能体生成引擎（@miaoma/agent-core）

- 基于 Codex CLI 的完整多智能体编排系统，包含 1 个 Coordinator 协调者 + 5 个 Collaborator 执行者的角色分工体系，支持按区域并行生成设计内容
- 完整的 Generation Run 状态机实现，覆盖 queued → preparing → designing → validating → repairing → completed / failed 全生命周期，支持状态持久化与断点恢复
- Codex Provider 抽象层，统一封装子进程调用、JSONL 事件流解析、Thread 会话管理，支持不同 AI 引擎的可插拔替换
- 实时流式输出能力，规划区域与生成进度实时同步到编辑器，支持光标指示器与增量渲染

设计 DSL 数据内核（@miaoma/design-schema）

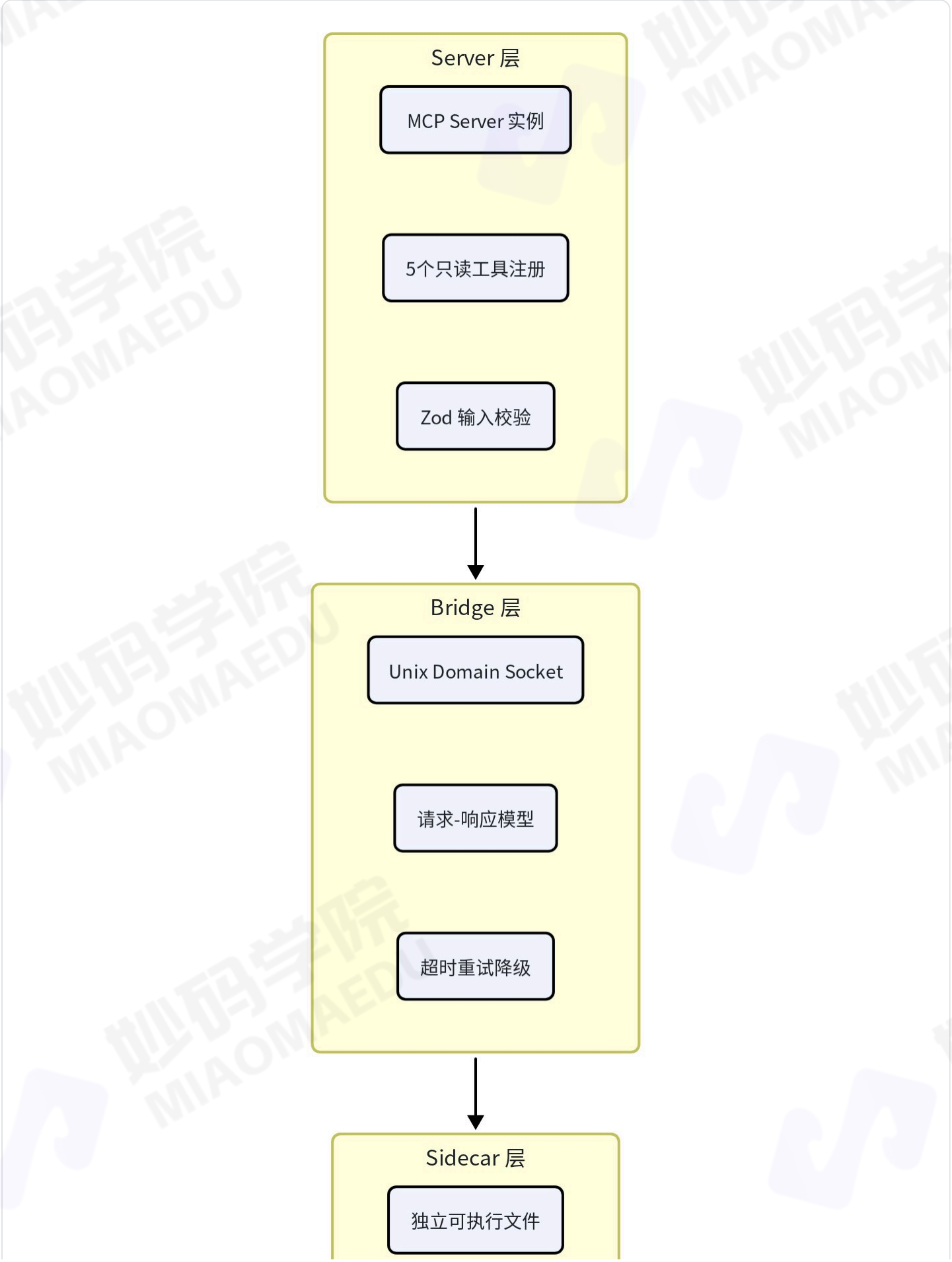
- 基于 Zod 的完整设计文档数据模型，采用可辨识联合类型设计，支持 Frame、Rectangle、Ellipse、Icon、Text 五大核心节点类型
- 正交属性拆分设计：布局属性、视觉样式、变换属性分层定义，原子化样式单元自由组合
- 设计变量系统（Design Variables）：颜色、字号、间距、圆角等设计 Token 统一管理，全局引用保证一致性
- 版本字段与向前兼容设计（documentVersion），支持 Schema 演进与数据迁移

类 Figma 可视化编辑器（Electron 桌面应用）

- 基于 Electron + React 19 + TypeScript + Vite 的完整桌面端设计编辑器
- 三栏经典布局：左栏图层面板 + 中栏画布编辑区 + 右栏属性配置面板
- 完整画布交互系统：三层坐标变换、命中测试、选择集管理、拖拽移动、缩放平移
- 命令模式 + 不可变数据 + 发布订阅三大编辑器范式，支持历史栈撤销重做

- 注册表驱动的属性面板，按节点类型动态渲染属性编辑器，支持批量编辑

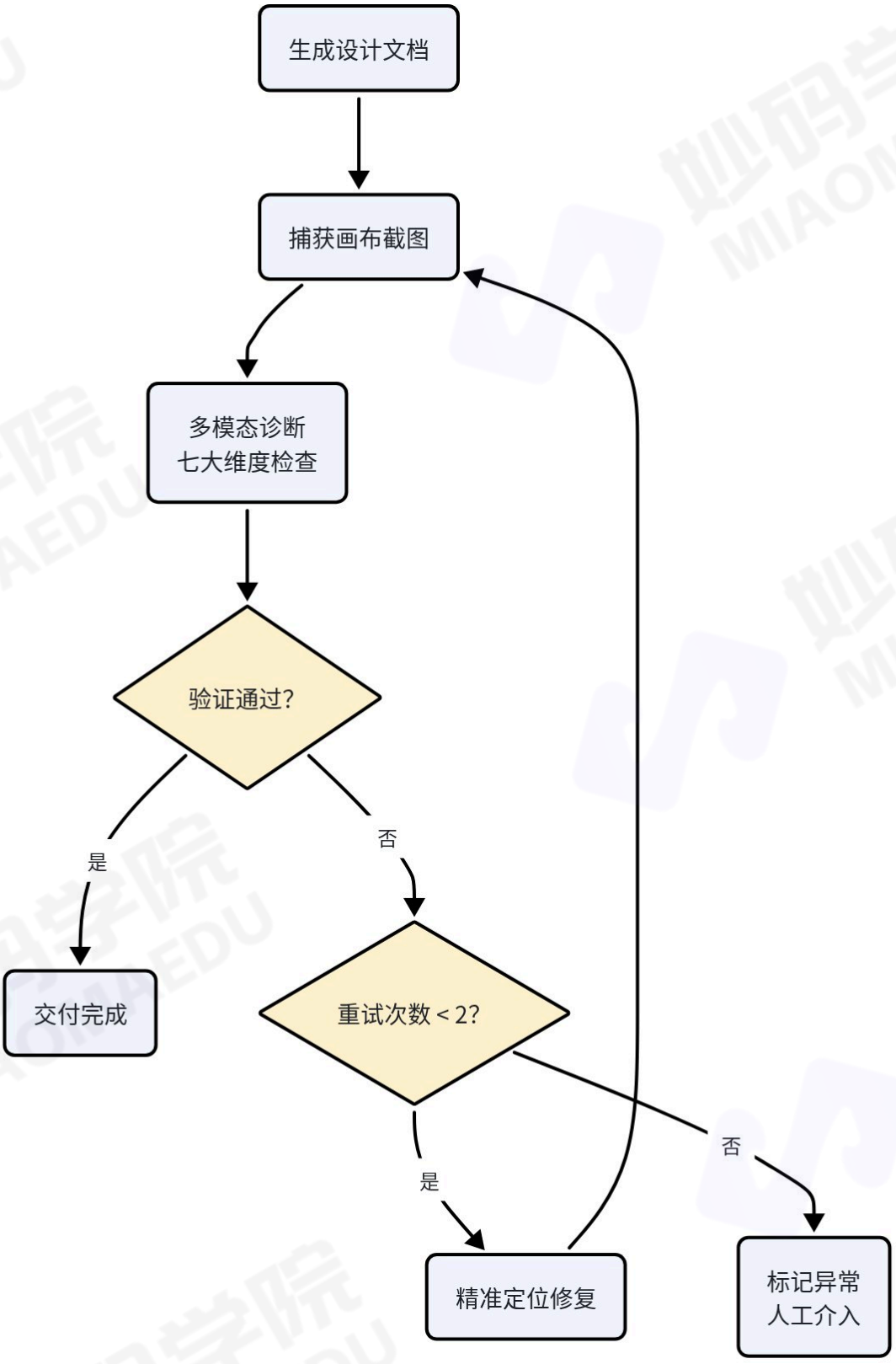
MCP 设计资产接入服务（@miaoma/mcp）





- 符合 Model Context Protocol 标准的只读 MCP Server，封装 5 大工具：get_app_state、get_selected_node、get_nodes、get_screenshot、get_assets
- Bridge 跨进程桥接层，通过 Unix Domain Socket（macOS）与 Named Pipe（Windows）实现 MCP 独立进程与渲染进程的可靠通信
- Sidecar 独立可执行文件模式，支持 Codex / Claude Code / Cursor 等所有 MCP 客户端启动
- 9 种分级错误码体系，超时控制、优雅降级、故障隔离等可靠性保障机制
- 资产目录化机制：自动提取设计文档中的图片资源，通过 miaoma-asset:// URI 协议统一引用

Visual Harness 视觉验证系统



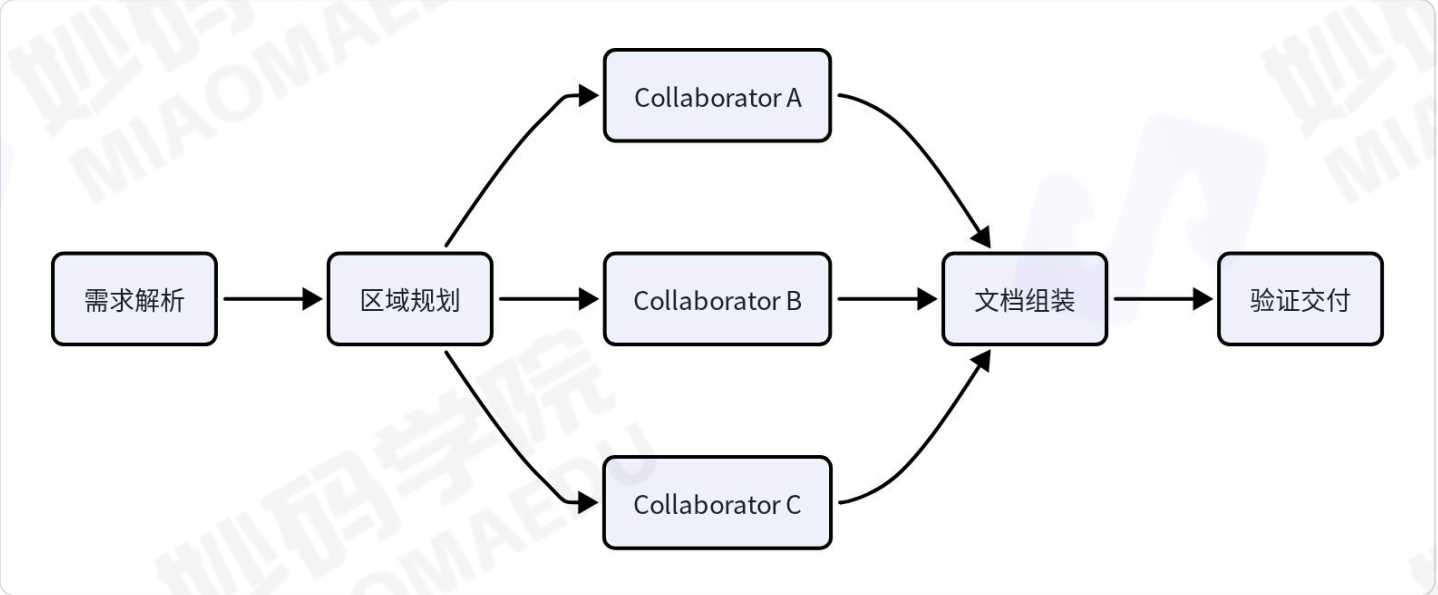
- Schema 结构校验 + 多模态视觉验证的双重质量保障体系
- 七大检查维度：布局对齐、层级关系、文本可读性、色彩对比度、组件完整性、变量一致性、边界溢出
- 自动修复闭环：发现问题 → 定位节点 → 精准修复 → 回归验证，最多重试 2 次防止无限循环
- 最小改动原则：每次修复只修改指定节点，不影响其他区域，控制变更范围

- 结构化 Visual Issue 描述：精确到节点 ID 的问题定位，修复目标明确可追溯

生成编排器（Orchestrator）

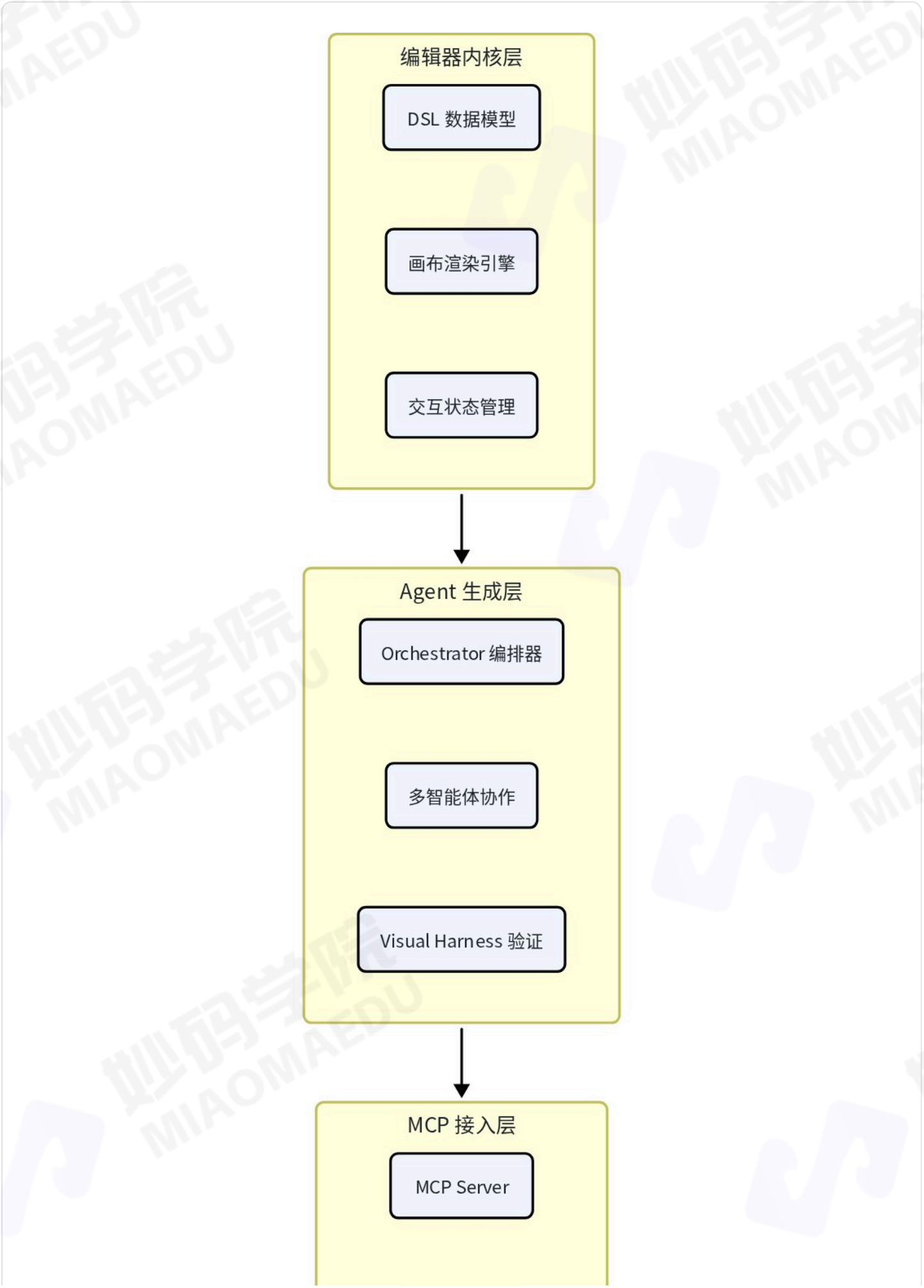
端到端生成流程的总指挥，负责将用户的自然语言需求拆解为可执行的生成任务，协调多智能体并行产出，并串联视觉验证闭环，最终交付完整可用的设计文档。是连接用户输入与设计产出的核心调度层。

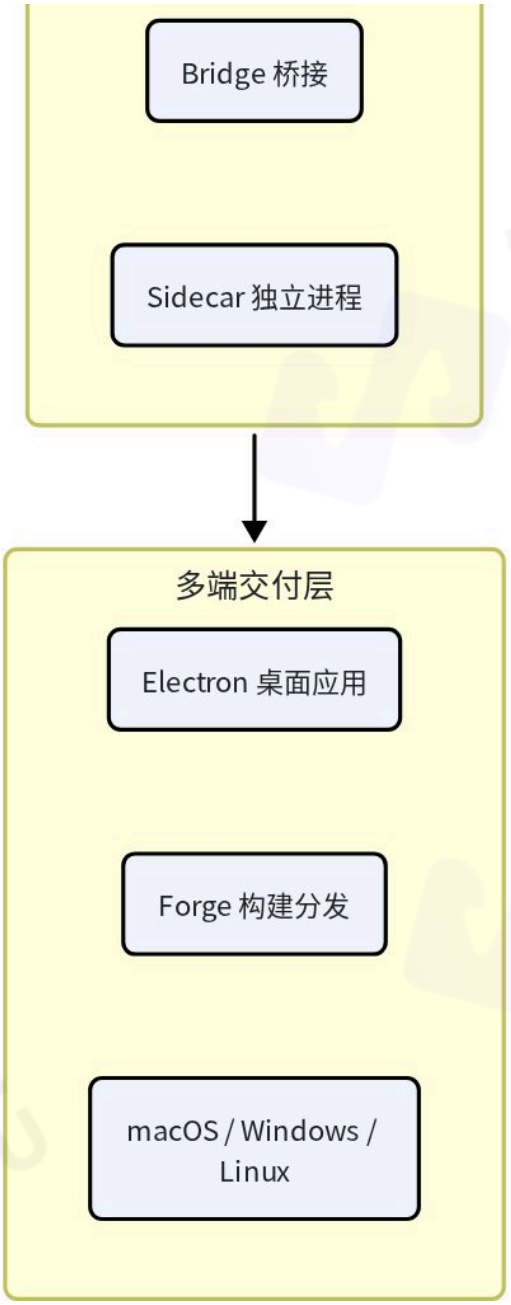
- **五阶段生成流水线**：需求解析 → 区域规划 → 并行生成 → 文档组装 → 验证交付，每个阶段有明确的输入输出与验收标准，流水线式推进保障生成过程可控可追溯
- **Coordinator 智能任务拆分**：根据页面描述自动识别功能区域，按导航栏、Hero 区、功能模块、页脚等语义单元划分生成边界，每个区域分配独立的 Collaborator 并行处理，拆分粒度兼顾并行效率与上下文完整性
- **并发调度与资源控制**：默认最多 5 个 Agent 并行执行，通过 MIAOMA_MAX_PARALLEL_COLLABORATORS 环境变量动态调整；采用信号量机制控制并发上限，避免资源耗尽导致整体性能下降
- **Visual Harness 验证闭环集成**：生成完成后自动调用 Harness 执行视觉自验证，七大维度逐项排查；验证不通过则触发修复循环，最多重试 2 次，在质量保障与执行效率之间取得平衡
- **统一事件总线与状态分发**：生成过程中的所有状态变更、进度更新、错误信息、流式增量通过事件系统统一分发，前端编辑器实时订阅并渲染，支持生成过程可视化与随时中断
- **乐观锁版本控制**：多 Agent 并发写入同一文档时，通过 revision 版本号实现乐观锁并发控制，检测到冲突时自动合并或重试，保障最终文档数据一致性



技术选型

整体架构分层设计


编辑器内核层
DSL 数据模型
画布渲染引擎
交互状态管理
Agent 生成层
Orchestrator 编排器
多智能体协作
Visual Harness 验证
MCP 接入层
MCP Server



本项目采用经典的**四层分层架构**，每层职责清晰，依赖方向明确，符合单一职责原则与单向依赖原则：

分层	职责范围	核心技术选型	设计思想
多端交付层	桌面应用、打包分发、安全加固	Electron + Electron Forge + Fuses	跨平台一次开发多端交付，纵深防御安全体系
MCP 接入层	标准化工具协议、跨进程桥接、Sidecar	@modelcontextprotocol/sdk + net.Socket	标准化协议解耦，一次接入多 Agent 兼容
Agent 生成层	多智能体协作、Harness 内核、视觉验证	Codex CLI + Zod Schema + 多模态验证	规划执行分离，契约驱动开发，自验证闭环

编辑器内核层	画布渲染、图层管理、DSL 数据模型	Canvas + React + Zod + Kiwi Schema	命令模式、不可变数据、发布订阅三大范式

详细技术选型说明

工程与包管理

- **包管理器：pnpm + workspace**
  硬链接共享依赖节省磁盘空间，安装速度快，Monorepo 原生支持良好；symlink 严格模式防止幽灵依赖，保证依赖声明的严谨性
- **Monorepo 架构：pnpm workspace**
  为什么选 Monorepo：核心引擎包和桌面应用放在同一仓库，版本对齐方便，跨模块修改一次性提交，文档代码同步维护。分层原则：packages 放可复用核心库（纯逻辑、无 UI 依赖），apps 放应用入口（依赖 packages），依赖从下往上，严格单向不循环

桌面端框架

- **应用框架：Electron**
  对比选型：vs Tauri — Electron 生态更成熟，Node.js 原生模块支持更好，本项目依赖 Sidecar 子进程和本地 Socket 通信，Electron 的 Node 环境更适配；vs 原生开发 — Electron 复用前端技术栈，一人全栈可以搞定，开发效率高，满足当前项目阶段需求。安全模型：主进程拥有系统权限，渲染进程沙箱隔离，预加载脚本桥接受控 API，遵循最小权限原则
- **构建工具：Vite（渲染进程）+ Electron Forge（打包分发）**
  Vite 热更新快，开发体验好，支持 main / preload / renderer 三入口独立构建；Electron Forge 一站式处理打包、签名、公证、发布，Maker 生态完善，减少配置工作量

前端与编辑器

- **前端框架：React 19 + TypeScript**
  React 19 带来最新的并发特性和 Hooks 优化，适合编辑器复杂状态管理场景；TypeScript 提供静态类型检查，配合 Zod 做运行时校验，双重保障类型安全
- **样式体系：TailwindCSS 4.0**
  CSS-first 配置，编译更快体积更小，原子类 CSS 开发效率高，与设计变量系统天然契合
- **画布渲染：Canvas 2D**
  相比 DOM 渲染，Canvas 在大量图层场景下性能更优，支持精确的像素级控制，适合专业设计工具场景；分层渲染优化重绘范围，提升交互流畅度

AI 与智能体

- **AI 引擎：Codex CLI**
  核心优势：本地沙箱环境拥有文件系统访问能力、Thread 持久化支持长任务、MCP 原生支持配置即用、内置 Harness 规划执行能力。对比 OpenAI API：API 无本地环境、无状态管理、MCP 需要自己实现，适合纯云端场景；CLI 更适合本地设计工具这种需要操作本地文件的场景
- **多智能体架构：Coordinator + Collaborator 模式**
  Coordinator 负责任务拆分和结果汇总，Collaborator 负责具体区域的设计生成，按区域并行执行提升生成速度。对比单 Agent：多 Agent 并行生成更快，职责更聚焦，但需要解决一致性控制和冲突合并问题
- **数据校验与结构化：Zod**
  一处定义多处复用：Schema 定义一次，同时满足 TypeScript 类型推导、运行时校验、AI 结构化输出约束，完美匹配 AI 项目需求
- **结构化输出方案：JSON Schema 注入 + Zod 校验 + 自动重试**
  为什么选这个方案：相比纯 Prompt 约束，Schema 注入格式稳定性高很多；Zod 做最后一层兜底校验，失败自动重试，生产级可靠性有保障

MCP 协议层

- **协议标准：Model Context Protocol**
  对比 Function Calling：Function Calling 是厂商绑定的 API 能力，MCP 是开放协议，工具与模型完全解耦，支持跨进程跨语言，有完整的工具发现、资源、提示等能力。类比 USB-C，一次接入所有设备通用
- **传输方式：Stdio + 本地 Socket 桥接**
  Stdio 是 MCP 标准传输方式，简单可靠，所有 MCP 客户端都支持；本地 Socket 桥接解决 MCP 独立进程与渲染进程的通信问题，跨平台适配 Unix Domain Socket 和 Windows Named Pipe
- **部署模式：Sidecar 独立二进制**
  进程隔离：MCP Server 崩溃不影响主应用；独立分发：Codex 等客户端可以直接启动二进制；安全边界：符合 Electron 安全模型，权限分离清晰

设计 DSL 与数据模型

- **Schema 定义：Zod 可辨识联合**
  通过 type 字段区分 frame / rectangle / ellipse / icon / text 五大核心节点类型，每种节点拥有独立的属性集，类型收窄精确，扩展性好，新增节点类型不影响现有逻辑。一处定义同时获得 TypeScript 类型推导、运行时校验、AI 结构化输出约束三重价值
- **设计变量：Design Variables**
  统一管理颜色、字号、间距、圆角、阴影等设计 Token，全局引用保证一致性，是 AI 多 Agent 并行生成时保持风格统一的基础保障。变量支持覆盖与继承，适配深色模式、多主题等扩展场景

- **存储格式：JSON 文档**
  调试友好，Git 可对比，不需要特殊数据库，简单够用；配合 revision 版本号实现乐观锁并发控制。文档结构扁平清晰，version + fileToken + variables + children 四层，易于序列化与传输
- **正交属性拆分**
  布局属性、视觉属性、交互属性正交拆分，每种属性独立演化互不干扰。填充、描边、阴影等视觉属性以数组形式支持多层叠加，既符合设计工具的行业惯例，也为后续扩展预留了空间

项目架构

miaoma-agent-core（智能体核心引擎）

简介：项目的 AI 大脑，负责多智能体编排、任务调度、状态管理、Codex 集成，是整个生成能力的核心引擎。

- **Coordinator 协调者**：总控 Agent，负责任务拆解、区域划分、结果汇总、质量把关。接收用户自然语言需求，拆解为多个独立的生成区域，分配给不同的 Collaborator 并行执行
- **Collaborator 执行者**：5 个并行工作的子 Agent，每个负责一个具体区域的设计生成。角色命名采用科学家体系（Newton、Tesla、Maxwell、Curie、Faraday），各自独立工作互不干扰
- **Codex Provider 抽象层**：统一封装 Codex CLI 的子进程调用、JSONL 事件流解析、Thread 会话管理、MCP 配置加载。Provider 模式实现与具体 AI 引擎的解耦，未来可扩展其他 Agent 引擎
- **Generation Run 状态机**：管理生成任务的完整生命周期，queued → preparing → designing → validating → repairing → completed / failed / cancelled。状态持久化支持断点恢复与历史回溯
- **并发控制**：MIAOMA_MAX_PARALLEL_COLLABORATORS 默认 5 个并行，可动态调整，防止资源耗尽

miaoma-design-schema（设计 DSL 数据内核）

简介：项目的统一数据协议标准，一处定义多处复用，同时作为 TypeScript 类型、运行时校验、AI 输出约束、自动文档的单一真相源。

- **可辨识联合类型系统**：通过 type 字段区分 Frame、Rectangle、Ellipse、Icon、Text 五大核心节点类型，每种节点有独立的属性集，类型收窄精确，扩展性强
- **正交属性拆分**：布局属性（x/y/width/height）、视觉样式（fill/stroke/shadow）、变换属性（rotation/scale）分层定义，原子化样式单元自由组合，避免属性爆炸
- **Design Variables 设计变量**：colors、typography、spacing、radii 四大类设计 Token 统一管理，全局引用保证一致性，是多 Agent 并行生成时风格统一的基础保障
- **MiaomaDesignDocument 根模型**：version + fileToken + variables + children 四层结构，带版本号支持向前兼容与数据迁移

miaoma-mcp（MCP 设计资产接入服务）

简介：设计资产的标准化出口，通过 MCP 协议将编辑器内部的设计数据暴露给外部 AI Agent，一次接入兼容所有 MCP 客户端。

- **MCP Server 层**：基于 @modelcontextprotocol/sdk 构建标准 MCP Server，注册 5 个只读工具。所有工具标注 READ_ONLY_TOOL_ANNOTATIONS（readOnly / idempotent / destructiveHint / openWorldHint），帮助 Agent 理解工具语义
- **Bridge 桥接层**：TCP / Unix Socket 本地桥接，MCP Server 独立进程通过本地 Socket 与 Electron 主进程通信。单次请求-响应模式，10 秒超时，64MB 消息上限，9 种分级错误码
- **Sidecar 独立进程**：打包为独立可执行文件，Codex / Claude Code 等客户端可直接启动。通过 --bridge-endpoint 参数传入桥接地址，与主应用建立连接
- **资产目录化**：递归遍历节点树提取 fill / stroke 中的图片资产，替换为 miaoma-asset:// URI 协议，SHA256 生成 assetId 去重
- **混合内容返回**：get_screenshot 和 get_assets 返回 text + image 混合内容，元信息 + base64 图片一起返回，Agent 同时获得结构化数据和视觉参考

miaoma-design-generation（生成编排器）

简介：生成流程的总指挥，端到端编排从需求输入到设计产物交付的完整流水线，协调各个子系统协同工作。

- **Orchestrator 编排器**：五阶段流水线 — 需求解析 → 区域规划 → 并行生成 → 文档组装 → 验证交付。统一调度 Coordinator、Collaborator、Harness 各模块
- **Visual Harness 视觉验证**：validate（诊断问题）→ repair（精准修复）→ run（执行完整循环）三方法。七大检查维度，最多 2 次修复重试，最小改动原则控制变更范围
- **文档组装器**：将多个 Collaborator 的生成结果合并为完整设计文档，乐观锁 revision 控制并发写入，防止冲突
- **事件系统**：统一事件总线，生成进度、状态变更、错误信息统一分发，前端实时订阅展示

apps/desktop（Electron 桌面应用）

简介：用户直接使用的桌面端产品，基于 Electron 构建，提供完整的可视化编辑体验和 AI 生成交互界面。

- **三层进程架构**：主进程（窗口管理、文件系统、Sidecar 桥接）+ 预加载脚本（contextBridge 安全桥接）+ 渲染进程（React UI、编辑器、画布渲染）。严格遵循最小权限原则
- **三入口构建**：VitePlugin 配置 main / preload / renderer 三个入口独立构建，各自的依赖和构建优化互不干扰
- **编辑器三栏布局**：左栏图层面板（树形图层管理）+ 中栏画布编辑区（Canvas 渲染 + 交互系统）+ 右栏属性面板（注册表驱动的动态属性编辑）

- **MCP 运行时**：主进程启动 Bridge Server，管理 Sidecar 生命周期，截图捕获、资产提取、节点查询等能力通过 IPC 通道提供给 MCP 层
- **extraResource 原生资源**：schemas 目录 + miaoma-mcp/bin 目录作为额外资源打包，不打进 ASAR 包，运行时通过 process.resourcesPath 动态解析路径

简历描述

初级 / 中级

AI 设计生成工具前端 / 全栈开发

- **技术栈**：TypeScript, React 19, Electron, TailwindCSS 4.0, Zod, Codex CLI, MCP
- **项目简介**：独立开发了一款桌面端 AI 设计生成工具，用户输入自然语言描述即可自动生成完整页面设计稿，支持类 Figma 可视化编辑、设计资产 MCP 标准化接入、一键导出前端代码，相比传统设计还原流程效率提升 5 倍以上，满足本地化隐私需求。
- **工作内容与成果**：
  - 基于 React 19 + Canvas 完成类 Figma 编辑器核心开发，实现图层面板、画布编辑区、属性面板三栏经典布局，支持元素选中、拖拽移动、缩放平移等核心交互，完整处理选中态、空状态、加载态等边界场景
  - 基于 Zod 设计完整的 DSL 数据模型，采用可辨识联合类型定义五大核心节点，实现设计变量系统统一管理颜色、字号、间距等 Token，作为 AI 生成的单一真相源，从数据层面保障输出格式一致性
  - 基于 @modelcontextprotocol/sdk 开发标准 MCP Server，封装 5 个只读设计资产工具，通过 Bridge 桥接层解决独立进程与渲染进程的跨平台通信问题，实现一次接入兼容 Codex / Claude Code 等多 Agent 平台
  - 基于 Electron Forge 完成跨平台打包配置，解决 Sidecar 二进制依赖的打包路径问题，正确处理开发环境和生产环境的资源路径差异，通过 Fuses 安全加固提升应用安全性，成功输出 macOS / Windows 双平台可运行安装包
  - 参与多智能体生成系统的前端对接，实现生成进度实时展示、流式增量渲染、光标指示器等交互，支持用户随时中断生成并保留中间结果

中级 / 高级

AI D2C 设计引擎全栈开发 / 架构设计

- **技术栈**：TypeScript, Electron, React 19, Codex CLI, Zod, MCP, Electron Forge, pnpm Monorepo

- **项目简介**：设计并开发了一款企业级桌面端 AI 设计生成产品，是面向设计与前端协作场景的**垂类设计智能体**。采用「编辑器内核 + Agent 生成引擎 + MCP 标准接入 + 多端交付」四层架构，支持自然语言输入一键生成完整页面设计，内置视觉自验证闭环保障产出质量，帮助团队将设计到代码的转化周期从天级缩短到分钟级。
- **工作内容与成果**：
  - **架构设计**：主导整体架构设计，采用 pnpm Monorepo 分层架构，清晰划分「编辑器内核层 / Agent 生成层 / MCP 接入层 / 多端交付层」四层职责。基于 Zod 实现全链路 Schema 契约驱动开发，从输入到生成到验证每一层都有格式保障，**有效规避 LLM 输出不稳定问题，将结构错误率降低 90% 以上**
  - **多智能体系统设计**：基于 Codex CLI 设计实现了 Coordinator + 多 Collaborator 的多智能体生成架构，按区域并行生成大幅提升速度；设计 Generation Run 状态机管理完整生命周期，支持断点续跑与历史回溯；引入 Visual Harness 视觉自验证机制，通过多模态诊断 + 自动修复循环保障设计质量，**将一次生成合格率从 60% 提升到 90% 以上**
  - **MCP 标准化接入**：主导设计资产 MCP 化方案，开发符合标准协议的只读 MCP Server，采用 Bridge 桥接模式解决 MCP 独立进程与渲染进程通信问题，实现 Sidecar 模式跨平台分发。一次接入兼容所有 MCP 客户端，**将 N×M 的集成复杂度降为 N+M**，为企业设计系统标准化接入奠定基础
  - **DSL 内核设计**：设计可演进的设计文档 DSL 模型，采用可辨识联合类型 + 正交属性拆分 + 设计变量系统的三层设计，支持向前兼容与数据迁移。建立 guard 函数 + diagnostics 诊断的防御式编程体系，保障数据内核的健壮性与可维护性
  - **工程化与多端交付**：搭建 Electron Forge 多平台构建流水线，集成 Vite 三入口构建、extraResource 原生资源打包、Fuses 安全加固。设计开发与生产环境的动态路径解析方案，解决 Sidecar 二进制分发难题，**支持 macOS / Windows / Linux 三端分发**，建立纵深防御的安全体系
  - **编辑器内核架构**：基于命令模式、不可变数据、发布订阅三大范式设计编辑器内核架构，实现 CQRS 命令查询分离、结构共享写时复制、事务打包等进阶机制。画布分层渲染优化交互性能，支持复杂设计文档的流畅编辑体验

课程内容

1. 需求分析、项目架构设计智能体执行流程与 AI Coding 实践

💡 初中级

- **D2C 技术全景与选型认知**：理解还原度的本质限制与信息论天花板，掌握直接生成、中间层 DSL、组件映射三条技术路线的差异与适用场景，理解 DSL 中间层作为防腐层的架构价值，能够初步判断不同业务场景的合适技术路线。
- **AI 生成链路核心设计模式**：掌握管道过滤器架构、适配器归一化、契约式防御编程、Schema 驱动开发四大生成链路核心模式，理解 T2UI 文本生成与 D2C 设计稿解析两条链路的异同与共通抽象。
- **编辑器内核三大范式**：深入理解命令模式、不可变数据、发布订阅三大编辑器核心范式的原理、实现方式与工程价值，掌握 CQRS、结构共享、事件通知等衍生机制的基本思想。
- **桌面端工程架构与分层思想**：理解 Electron 三层进程模型与沙箱安全机制，掌握 IPC 通信的通道规范与边界约束，理解 Monorepo 多包分层、单向依赖、按业务域划分模块的架构原则。

高级

- **Agent 架构设计与生成链路编排**：掌握 L1 单轮到 L4 多智能体的四个演进阶梯与复杂度跃迁，能够设计规划执行分离的 Agent 五层架构，独立编排完整的 T2UI 生成流与 D2C 解析管道，具备多 Agent 协作方案的评审与决策能力。
- **编辑器内核架构设计与性能优化**：能够基于命令模式、不可变数据、发布订阅三大范式设计完整的编辑器内核架构，掌握 CQRS 命令查询分离、结构共享写时复制、事务回放与宏录制等进阶机制，具备 Selector 缓存优化、防抖节流、指纹比对等性能调优能力。
- **桌面端工程架构设计与质量保障**：能够设计分层清晰、单向依赖的 Monorepo 多包工程架构，掌握 Electron 沙箱安全模型与 IPC 通道命名规范，具备按业务域划分模块、设计防御式编程边界、保障长期可维护性的工程质量把控能力。

2. 可视化编辑器全栈开发、Agent 面板、 DSL 数据内核层设计

💡 初中级目标

- **理解左栏导航、中栏创作、右栏配置的经典三栏范式**：掌握 Agent 面板消息结构与图层面板树形视图的核心交互逻辑，建立专业设计工具的信息架构认知。
- **掌握画布坐标系统与基础拖拽交互**：掌握屏幕/视口/世界三层坐标的变换关系，理解命中测试、选择集管理、reducer 状态机的实现原理，能够基于 miaoma-editor-interaction 包实现基础的元素选中与拖拽移动。
- **了解DSL 节点类型系统与样式抽象**：理解 JSON Schema DSL 作为中间层的解耦价值，掌握 frame/rectangle/ellipse/icon/text 五种核心节点类型与正交属性拆分原则，了解填充、描边、阴影等原子化样式单元的数据结构。
- **了解 Figma 资产导入 ETL 流水线架构**：了解设计资产导入的生态价值与工程意义，掌握 Extract → Transform → Load 三步流水线架构，理解适配器模式在格式转换中的应用。

高级目标

- **深入编辑器内核三大范式与历史栈**：深入掌握命令模式、不可变数据、发布订阅三大编辑器核心范式的原理与工程价值，理解事务打包、历史栈指针、结构共享等进阶机制的实现思路。
- **深入画布交互状态机与分层渲染架构**：深入理解 reducer + draft 的交互状态机设计，掌握草稿预览与事务提交的分离机制，能够拓展设计吸附对齐算法与多层 Canvas 分层渲染架构。
- 深入理解 INSPECTOR_PROPERTY_REGISTRY 注册表的配置驱动设计思想，掌握节点类型过滤、分组渲染的实现方式，具备批量编辑冲突合并策略的设计能力。
- **深入DSL 内核设计与运行时防御体系**：深入掌握可辨识联合、正交属性拆分、原子化样式组合等 DSL 核心设计原则，理解 guard 函数 + diagnostics 诊断系统的防御式编程实践，具备数据内核质量保障体系的设计能力。

3. 基于 Codex 多智能体引擎、Harness 内核架构与设计产物自验证落地

💡 初中级目标

- **理解 Codex 本地集成方案**：掌握子进程调用模式与 JSONL 事件流解析，建立 AI Agent 与本地 IDE 协作的工程化认知，理解为何选择端侧 Agent 而非纯 SaaS API。
- **掌握多智能体协作架构**：理解 Coordinator + Collaborator 的角色分工，掌握任务按区域拆分与并行执行的编排模式，建立多 Agent 系统的设计思维。
- **掌握 Harness 内核三阶段工作流**：记忆（历史持久化）、规划（任务拆解）、实施（分片生成与文档组装）的核心数据流与状态流转。
- **理解视觉自验证闭环机制**：掌握「截图捕获 → 多模态诊断 → 精准修复」的完整循环，建立 AI 产出质量自动化保障的工程思维。
- **掌握 JSON Schema 契约驱动开发**：理解结构化输出的必要性，掌握 Schema 定义、原生注入与容错解析的落地方法，建立防御式编程意识。

高级目标

- **架构设计**：能够独立设计 Codex Provider 抽象层，支持不同 AI 引擎的可插拔替换，具备 Provider 模式的工程化落地能力。
- **掌握多智能体编排能力**：掌握多智能体任务分配算法，能够基于区域复杂度动态调整协作策略，设计任务优先级与资源调度机制。
- **内核扩展**：能够为 Harness 新增记忆维度（风格记忆、组件库记忆）与中间件钩子，具备内核二次开发的架构视野。

- **深入验证体系：**能够设计多层验证策略（Schema 层 + 视觉层 + 语义层），建立分级修复与降级机制，构建完整的质量保障体系。

4. MCP 设计资产接入、服务端开发与多端交付及面试实战复盘

💡 初中级目标

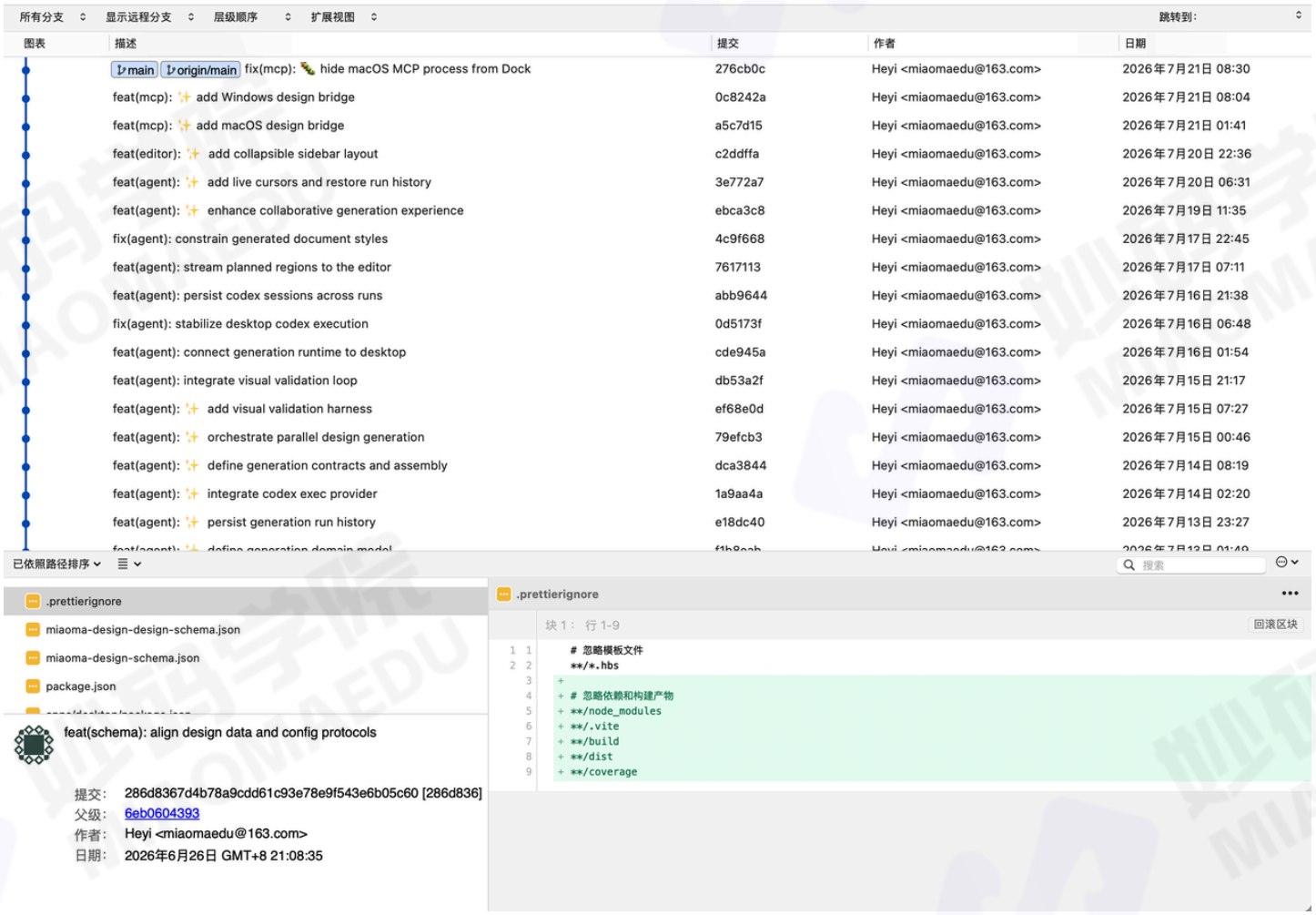
- **理解 MCP 化价值与设计资产抽象思路**：掌握将设计资产抽象为标准化领域模型的方法论，理解 MCP 协议如何将 N×M 集成复杂度降为 N+M，能结合业务场景判断是否适合 MCP 化改造
- **具备 MCP Server 开发与工具注册能力**：能基于 @modelcontextprotocol/sdk 从零搭建标准 MCP Server，熟练使用 registerTool 注册工具、定义 Zod 输入 Schema、处理 text+image 混合内容返回
- **理解跨进程桥接架构与 IPC 通信方案**：掌握 Bridge 模式解决 MCP 独立进程与渲染进程通信的核心设计思路，理解 Unix Domain Socket 与 Windows Named Pipe 的技术差异及跨平台适配方案
- **掌握设计到代码的完整还原工作流**：理解从结构化设计 JSON + 多模态截图参考生成 React 组件的全链路，能梳理 Read-Reference-Implement 三步法并解释各环节的输入输出与价值
- **了解 Electron 多端构建与安全分发流程**：掌握 Electron Forge 构建体系的三入口构建、extraResource 资源打包、Fuses 安全加固，能说清 macOS 与 Windows 打包的产物差异和常见踩坑点

高级目标

- **D2C 项目面试体系化输出**：能从架构设计、AI 工程化、标准化能力、跨端交付、工程质量五个维度提炼简历亮点；掌握协议原理、架构选型、质量保障、性能优化四类高频问题的回答框架，讲清技术决策背后的 trade-off
- **MCP 协议架构设计决策能力**：不仅会用 SDK 搭 Server，还能讲清 MCP 与 Function Calling、LSP 的本质差异；能根据业务场景判断何时只读、何时开放写入，理解 Tool Annotations 对 Agent 调度策略的影响
- **跨进程 IPC 可靠性工程能力**：掌握本地 Socket 桥接的超时、重试、错误分级、优雅降级全套设计；能对比 Unix Domain Socket 与 Named Pipe 的平台差异，解释为什么 Bridge 层采用单次连接模式而非长连接
- **Sidecar 模式跨平台编译与分发**：理解 Sidecar 独立进程的架构价值，掌握 macOS / Windows 双平台二进制构建策略；能解释为什么 Sidecar 必须走 extraResource 而非打进 ASAR，以及开发与生产环境的路径差异处理

- **Electron CI/CD 构建体系搭建**：能独立搭建多平台构建流水线，理解 package 与 make 的分层职责；掌握 Fuses 安全加固、代码签名、公证、自动更新的完整分发链路，具备纵深防御的安全设计思维

代码 commit 截图



“企业级项目”指采用企业常用技术栈及开发流程的教学模拟项目，不等同于商业生产环境使用的项目